

Forensic APFS File Recovery

Jonas Plum

Siemens AG

München

jonas.plum@siemens.com

Andreas Dewald

ERNW Research GmbH

Heidelberg

adewald@ernw.de

ABSTRACT

In forensic computing, especially in the field of postmortem file system forensics, the reconstruction of lost or deleted files plays a major role. The techniques that can be applied to this end strongly depend on the specifics of the file system in question. Various file systems are already well-investigated, such as FAT16/32, NTFS for Microsoft Windows systems and Ext2/3/4 as the most common Linux file systems and HFS/HFS+ for macOS. There also exist tools, such as the famous Sleuthkit by Brian Carrier that provide file recovery features for those file systems by interpreting the file system's internal data structures. APFS is the new file system for Apple devices that is applied by default on all current iOS mobile devices, as well as macOS since *High Sierra*, and is thus currently rolled out on a large number of devices. However, for APFS, no forensic file recovery methodologies have been developed so far. In this paper, we propose different approaches to identify and recover (deleted) files on an APFS file system. We implemented our approaches as a proof of concept tool and evaluate those against each other and against file carving.

CCS CONCEPTS

• **Applied computing** → **Data recovery**; *Computer forensics*; *Evidence collection, storage and analysis*;

KEYWORDS

Digital forensics, Data recovery, File systems, APFS, Open Source, Tool

ACM Reference Format:

Jonas Plum and Andreas Dewald. 2018. Forensic APFS File Recovery. In *ARES 2018: International Conference on Availability, Reliability and Security, August 27–30, 2018, Hamburg, Germany*. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3230833.3232808>

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

ARES 2018, August 27–30, 2018, Hamburg, Germany

© 2018 Copyright held by the owner/author(s).

ACM ISBN 978-1-4503-6448-5/18/08.

<https://doi.org/10.1145/3230833.3232808>

1 INTRODUCTION

Persistent storage devices are still one of the most important sources of digital evidence in digital investigations. In particular, the forensically sound reconstruction of deleted files alongside with their metadata is an important step in the forensic process. To this end, usually two distinct kinds of methods are applied: file carving or file system parsing. File carving methods try to identify files content of different file formats by common patterns (magic bytes). The advantage of those methods is that the underlying file system that was used to store the files is not relevant and does not need to be understood. Therefore, carving still works for newly introduced and unknown file systems. However, this comes with several disadvantages: First, only file types for which well-known patterns exist can be reconstructed. Further, besides special approaches for specific file types, file carving is mostly not able to handle file fragmentation. And last, but in the forensic context especially important, file carving is usually not able to obtain metadata, such as timestamps or even file names and directory structures, as this information is only held in the internal structures of the file system. On the other hand, parsing of file system data provides all this information, although file recovery possibilities can be limited, depending on the particular file system. The major problem of file system parsing, however, is that for each upcoming file system, old tools do not provide any results at all, unless specific methods for file recovery have been developed and implemented for a new file system.

1.1 Motivation

There exist forensic file recovery approaches and tools for almost all common file systems, such as FAT 16/32, NTFS, Ext2/3/4, HFS, and HFS+. Apple recently introduced their new file system APFS (Apple File System), which is rolled out on all current iOS and macOS devices. So there is a need for the development of new forensic file recovery approaches.

In APFS two stages of deleted files exist. Normal deleted files might still be referenced from versioning structures in the file system. Permanently deleted files are not referenced from the file system any more. While parsing can restore normal deleted files, our goal is to restore permanently deleted files as well. To the best of our knowledge, there is no (publicly available) tool for forensic recovery of permanently

deleted files on APFS volumes. We want to address those problems as outlined in the next section.

1.2 Contributions

In this paper, we make the following contributions:

- We propose different methodologies for forensic file recovery for APFS that leverage the knowledge of APFS-specific internals.
- We present AFRO, our open source implementation of a forensic file recovery tool for APFS, implementing all the presented methodologies.
- We describe our APFS forensic tool evaluation dataset that we used to benchmark our approaches, including ground truth documentation, that we make publicly available alongside with this paper.
- We compare our different novel approaches amongst each other and compare our tool AFRO with existing approaches.

1.3 Related Work

Apple Inc [2] released information about some features of APFS. However, this information contains not much information about internal structures of the file system. Hansen and Toolan [9] describe some artifacts and structures of APFS, of which we updated and completed in our whitepaper [5].

Some Open Source projects for parsing APFS exist. There is an experimental APFS fuse driver Gander [7] and a collection of libraries to mount, dump and analyze APFS volumes and containers [3]. Neither of those support recovery of permanently deleted files. There is also commercial software like Recovery Explorer [13], Biskus APFS Capture [14] and APFS for Windows by Paragon Software Paragon Technologie GmbH [12]. Some of this software claims to be able to recover files, but their methodologies are not public.

1.4 Outline

The remainder of this paper is structured as follows: Section 2 describes parsing of the APFS file system. In Section 3, we suggest different methodologies for file recovery using the file system metadata for APFS and we present AFRO, a prototype implementation of our methods. For evaluating different file recovery approaches, we generated an evaluation dataset, which we make publicly available alongside with this paper, and for which Section 4 describes the details of the dataset, how it has been generated, as well as the ground truth of the different images in the corpus. Section 5 presents the evaluation results of AFRO and a comparison of the different methodologies that have been implemented. In Section 6, we finally discuss the evaluation results, identify limitations and future work and draw a conclusion.

2 APFS PARSING

In their article Hansen and Toolan [9] provide a description of many APFS data structures, which was further completed and updated in our earlier publication [5]. The following parsing process is based on those papers and describes a method to extract files from APFS. The aim of parsing in this paper is to extract a complete file hierarchy. For a more detailed description of the APFS internals refer to Hansen and Toolan [9] and Dewald and Plum [5].

By design, APFS offers the ability to recover certain states of the file system including old or deleted versions of files. The container superblock contains a reference to the preceding container superblock, which contains information in an older state of the file system. In this way multiple older states can be recovered by parsing this chain of container superblocks. The following list describes the steps necessary for parsing APFS including the older states of files:

- Step 1** Parse the container superblock
- Step 2** Parse the container OMAP
- Step 3** Parse the container OMAP nodes
- Step 4** Parse the volume superblock
- Step 5** Parse the volume OMAP
- Step 6** Parse the volume OMAP nodes
- Step 7** Parse the root tree
- Step 8** Parse previous container superblocks

The relevant fields and function of the listed structures is described in the next sections. The Steps 4 - 7 are repeated for every volume. Steps 1 - 7 are repeated for every container superblock. The listed steps of the parsing process are connected to the layout of APFS in Figure 1.

In the next section, the structure of APFS object headers is given. Hereafter the listed steps of the APFS parsing process are described in detail.

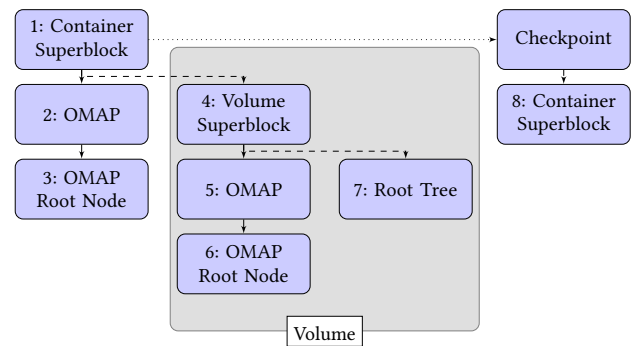


Figure 1: Connection between Steps of the APFS parsing process

2.1 Object Header

File system data in APFS is stored as objects. All objects have a 32-byte header. Table 1 displays the relevant fields of this header.

pos	size	type	id
0	8	u8le	cksum
8	8	u8le	oid
16	8	u8le	xid
24	2	u2le	type
28	2	u2le	subtype

Table 1: Relevant Fields in the Object Header

The first eight bytes of the header are a variant of the Fletcher’s checksum (*cksum*) as described by Kodis [10]. The original algorithm was adapted to 64bit input. The checksum is calculated using the data of the block of the object without the first 8 byte and is described in detail in our whitepaper [5]. The checksum is followed by the *oid* of the object which is used for referencing objects. The *xid* describes the version of an object. It is incremented when the object is changed. The next two bytes describe the *type* of the object. Object types relevant for file system parsing are:

0x1 Container Superblock	0xB B-Tree
0x2 Rootnode	0xC Checkpoint
0x3 Node	0xD Volume Superblock

The *subtype* is used to further distinguish types of rootnodes and nodes.

2.2 Step 1: Parse the container superblock

APFS is structured in a single container that can contain multiple APFS volumes. The container superblock is the entry point to the file system and is located in the first block of the file system. This does not apply for improperly unmounted file systems, but handling of those is not in scope of this paper. The fields of the container superblock which are relevant for parsing are listed in Table 2.

The container superblock starts, like every other file system object with the object header which we described in the last section. After the object header every container starts with the four *magic* bytes ‘NXSB’. The second field is the *block_size* of the file system, which is used for converting block offsets in byte offsets. The *xp_desc_index*, *xp_desc_base* and *xp_desc_index_len* fields can be used to find older container superblocks. This procedure is described in Step 8 in Section 2.9. The *omap_oid* field contains an offset in blocks to the OMAP (object map) which is needed in the next Step.

The oids of all volumes are stored in the superblock in *fs_oid* with the length of this list stored in *max_file_systems*.

pos	size	type	id
0	4	str	magic ‘NXSB’
4	4	u4le	block_size
80	8	u4le	xp_desc_base
104	4	u4le	xp_desc_index
108	4	u4le	xp_desc_index_len
128	8	u8le	omap_oid
152	4	u4le	max_file_systems
160	...	u8le	fs_oid

Table 2: Relevant Fields in the Container Superblock

2.3 Step 2: Parse the container OMAP

The container superblock references an OMAP (object map) which is an APFS B-tree. B-trees manage one or more nodes. The structure of the B-tree is quite simple, as shown in Table 3. There are currently two known *btree_types*: 0 for B-trees in volumes and 1 for B-trees in the container. Both types contain the block offset of the *root* node, which is processed in Step 3.

pos	size	type	id
0	8	u8le	btree_type
16	8	u8le	root

Table 3: B-Tree Structure

2.4 Step 3: Parse the container OMAP nodes

The nodes of the OMAP B-tree only contain OMAP entries, but they are parsed equally to other nodes in the volumes, therefore nodes and the general parsing of node entries are described in the following sections, while only OMAP entries are explained in detail.

Nodes and root nodes have the same internal structure. All nodes are flexible containers that are used for storing different kind of entries. They can be part of a B-tree or exist on their own. A node starts with a node header as described in Table 4. The *level* which describes the depth of the node in the B-tree. The level is zero for leaf nodes and > 0 for index nodes. The following attributes define the *entry_count*, the position of key section (*keys_offset* and *keys_length*) and data section *data_offset*. This node header information is followed by a list of entry headers that act as pointers to the entry

keys and entry values. This way for each entry the node contains an entry header at the beginning of the node, an entry key in the middle of the node and an entry value at the end of the node as displayed in Figure 2.

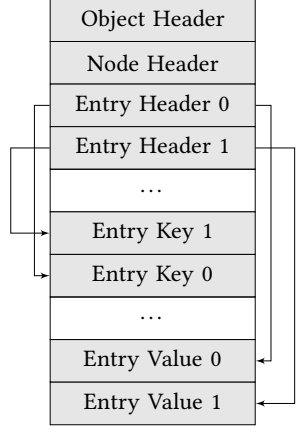


Figure 2: Internal Structure of Nodes

pos	size	type	id
2	2	u2le	level
4	4	u4le	entry_count
10	2	u2le	keys_offset
12	2	u2le	keys_length
14	2	u2le	data_offset
24	...	EntryHead	entry_heads
...	...	EntryKey	entry_keys
...	...	EntryValue	entry_values

Table 4: Node Structure

Nodes contain different kinds of entries. OMAP entries have the kind *0x0*, inodes entries *0x3* and extent entries *0x8*.

All entry types have the same entry header structure, but different key and value structure.

2.4.1 Entries. Once the node header is parsed the single entries in a node can be located and processed. The entry header structure is the same for all node entries and is defined in Table 5. It consists of the position for the entry key (*key_offset* and *key_size*) as well as the position of the related values (*val_offset* and *val_size*). While the node header contains the offset to all keys the *key_offset* in this header is especially for one key. The offset with regard to the object start is calculated with 32 (object header size) + 24 (node header size) + *keys_offset* (from the node header) + *key_offset* (from the entry header).

pos	size	type	id
0	2	s2le	key_offset
2	2	u2le	key_size
4	2	s2le	val_offset
6	2	u2le	val_size

Table 5: Entry Header Structure

2.4.2 Entry Keys. All entry keys start with an 8 byte field. The highest byte determines the *kind* as described in the list above, while the remaining bytes contain the *obj_id*. The *obj_id* contains the id of the referenced element. This can be a *file_id* or an objects *oid*. The *obj_id* can be followed by more fields depending on the entry kind. The entries are sorted in a node by their complete key which contains the kind and *obj_id* as well as the individual key content.

2.4.3 Pointer Value. Pointer values do have a special role. Nodes which are not leaf nodes (all nodes with level > 0) only contain an 8 byte pointer value. This is independent of the entry kind in the key. Pointer values are used for the structure of B-tree and reference other nodes

2.4.4 OMAP Entry. OMAP entries are used for mapping object oids to block offsets. In the container OMAP they map oids of volumes, which are stored in the *fs_oid* field of the container superblock to block offsets. The key contains the *kinds*, *obj_id* and the *xid* of the object as shown in Table 6. The OMAP entry value (Table 7) contains the position (*paddr* for physical address) and *size* for the object.

pos	size	type	id
0	8	u8le	kind & obj_id
8	8	u8le	xid

Table 6: OMAP Entry Key Structure

pos	size	type	id
4	4	u4le	size
8	8	u8le	paddr

Table 7: Relevant Fields in the OMAP Entry Value

After using the OMAP entries in the container superblock OMAP to map oids to the *fs_oid* to block offsets the volume superblocks at these offsets can be parsed.

2.5 Step 4: Parse the volume superblock

Every volume in an APFS container starts with a volume superblock. The four fields relevant for parsing in the volume superblock are listed in Table 8.

pos	size	type	id
0	4	str	magic 'APSB'
96	8	u8le	omap_oid
104	8	u8le	root_tree_oid
672	...	str	volname

Table 8: Relevant Fields in the Volume Superblock

The volume superblock starts, similarly to the container superblock, with the *magic* bytes 'APSB'. It also contains a reference to its own OMAP B-tree in *omap_oid*. The root of the file system is referenced in the *root_tree_oid* field. The last information stored in the volume superblock is the volume name in *volname*.

2.6 Step 5: Parse the volume OMAP

The parsing of the OMAP in the volume is done equally to the parsing of the OMAP in the container as described in Step 2.

2.7 Step 6: Parse the volume OMAP nodes

The nodes in the volumes OMAP are parsed the same way as the nodes in the container in Step 3.

2.8 Step 7: Parse the root tree

Using the OMAP of the volume the *root_tree_oid* in the volume superblock can be mapped to a block offset. The root tree is a node and works like the nodes described in Step 3. The root tree contains different kinds of entries which contain information about files and folders. The inode entries and the file extent entries can be used to extract all files with their metadata from the volume.

2.8.1 Inode Entry. The inode entry contains metadata about files and folders. It does not have any extra key content besides the *kind* and *obj_id*. The keys *obj_id* contains the *file_id* of the file for that the metadata is stored.

The inode entry value (Table 9) starts with a reference *parent_id* to the parent folders *file_id*. The following *extents_id* usually contains the *file_id*, but might contain an id that is different from the normal *file_id* if the file was cloned. It is used for matching inode entries with extent entries. The next four attributes are timestamps (*ctime*, *mtime*, and *atime*). The *xf_num_ext* field contains the count of the following extended fields. Extended fields contain additional information about files and folders, most importantly name and filesize are stored here. *xf_used_data* is the number of bytes for all extended fields. *xf_header* is a list of type and length values of the following extended fields, as shown in Table 10. The

pos	size	type	id
0	8	u8le	parent_id
8	8	u8le	extents_id
16	8	u8le	ctime
24	8	u8le	mtime
40	8	u8le	atime
92	2	u2le	xf_num_ext
94	2	u2le	xf_used_data
96	4*xf_num_ext	xf_header	xf_header
...	...	xf_field	xfields

Table 9: Relevant Fields in the Inode Entry Value

pos	size	type	id
0	2	u2le	type
2	2	u2le	length

Table 10: xfield header Structure

extended fields contain information based in their type, as follows:

0x204 name (string)

0x2008 size (u8le)

Extended fields are aligned at multiples of 8 byte, so padding between extended fields might exist.

2.8.2 File Extent Entry. File extent (*fext*) entries contain information about the position and size of file content. One file may have an arbitrary number of extents. The key (Table 11) starts with the *kind* and *obj_id*, where the *obj_id* is referenced by the *extents_id* field of the inode. These fields are followed by the 8-byte *offset* into the file data, so that multiple extent values are sorted in order. The extent entry value (Table 12) first contains the *len* of the extent. This is usually a multiple of the file systems *block_size*. The second field *phys_block_num* contains the offset of the extent in blocks.

pos	size	type	id
0	8	u8le	kind & obj_id
8	8	u8le	offset

Table 11: Extent Entry Key Structure

pos	size	type	id
0	8	u8le	len
8	8	u8le	phys_block_num

Table 12: Relevant Fields in the Extent Entry Value

2.9 Step 8: Parse previous container superblocks

When all inode and extent entries in all volumes are parsed the reference to older version of container superblocks in the container superblock can be used. Those older container superblocks are located in the checkpoint descriptor area. The *xp_desc_index* field points to the currently used checkpoint descriptor. This position is relative to the *xp_desc_base* offset. The *xp_desc_index_len* field contains the object count of the current checkpoint descriptor. While the checkpoint descriptor starts with an checkpoint object, this checkpoint object is followed by the older container superblock. Container superblocks chained this way can be processed to restore some deleted or changed files.

3 APFS FILE RECOVERY METHODOLOGIES

APFS is a copy-on-write file system and thus each block is copied before changes are applied. Therefore, a history of all files which were not overwritten and according file system structures exist. This results in a high number of artifacts that can be utilized in forensic file recovery.

From those artifacts, we identified three different approaches for file recovery that rely on different artifacts as entry points, which we present in this section: The signature-based file system metadata carving methods “NXSB carving” and “APSB carving”, as well as the heuristic file system metadata carving method “node carving”. All methods iterate the file system in blocks of 4096 bytes, which is the smallest block size observed in APFS. Those blocks are checked for metadata structures which in turn are parsed and used to extract files. Most of the file metadata information and the file name are included in inode values in the file system. Pointers to file content is stored in extents. To recover files, those two value types are the most valuable recoverable artifacts. We describe our three approaches in the subsequent sections.

3.1 Signature-based File System Metadata Carving

The container superblock and the volume superblock have a fixed signature directly preceding the object header. The container superblock contains the ASCII string “NXSB”, while the volume superblock contains the string “APSB”. Because both strings have a fixed offset within the block, these objects can easily be found when carving the file system. This way they can be found even when references from other structures have been deleted. Using this technique, it might be possible to recover files which are not referenced from the first block of the file system. The objects checksum can be validated to eliminate false positives. When the objects have



Figure 3: Logo for AFRO

been found, the standard parsing procedure can be followed from this point. The algorithm for the NXSB carving is the following:

Step 1 Iterate file system in blocks of 4096 byte

Step 2 Check if bytes 32 to 36 equal ‘NXSB’

Step 3 Check checksum

Step 4 Continue with parsing Step 1 as described in Section 2

The method for APSB carving is similar:

Step 1 Iterate file system in blocks of 4096 byte

Step 2 Check if bytes 32 to 36 equal ‘APSB’

Step 3 Check checksum

Step 4 Continue with parsing Step 4 as described in Section 2

Both procedures skip Step 8 from Section 2. In NXSB carving, older container superblock are found anyway, while the APSB carving does not carve container superblocks at all.

3.2 Heuristic File System Metadata Carving

Nodes do not have a specific signature. They can be carved using possible value combinations for the object type and the object subtype as an indicator. For verification and to prevent false positives, again the checksum can be used, like in the methods before. The algorithm for the node carving is the following:

Step 1 Iterate file system in blocks of 4096 byte

Step 2 Check if bytes 24 to 26 (object type) equal 0x2 or 0x3

Step 3 Check if bytes 28 to 30 (subtype) equal 0xe (Files subtype)

Step 4 Check checksum

Step 5 Parse node (Equals Step 7 as described in Section 2)

3.3 Implementation of AFRO

We implemented the approaches roughly outlined in Section 3 as an open source software prototype named AFRO (APFS File Recovery Options). Its logo can be seen in Figure 3. AFRO can be downloaded from <https://jonasplum.de/ln/afro>.

AFRO processes an APFS file system image in two stages. First, it extracts inode and extent values using one of the methods described before and as a second process, it links those values to each other and outputs information about the files.

Parsing the APFS structures is achieved using a parser generated from a Kaitai Struct file. Kaitai Struct defines a declarative language to describe binary data structures. This definitions are stored as .ksy files. For APFS, an apfs.ksy file was generated. The .ksy file can then be converted to parsers in different languages using the *kaitai-struct-compiler*. We converted the apfs.ksy file to a python APFS parser. Carving is implemented as reading blocks of a defined size and checking the different carving criteria on those. If a block matches the criteria, it is parsed using the APFS parser. With the APFS parser, all inode and extent values are extracted for the second stage, the processing.

In the processing stage, the extracted inode and file extent values are matched. Afterwards information of the matched values (item_type, xid, parent_id, item_id, volume, path, name, atime, mtime, ctime, size) is extracted and the md5 hash value of the file is calculated. All this information is stored in a csv file.

4 EVALUATION DATASET

We generated five different datasets to evaluate the effectiveness of the developed recovery methods. All datasets can be downloaded from https://jonasplum.de/ln/apfs_dataset.

4.1 Dataset Generation

To simulate usage and file system wear, 1000 actions are performed on the file system. The actions are randomly selected from all possible actions of the following list:

add_file A random file from the EDRM File Formats Data Set 1.0 [6] is added.

delete_file Remove a random file from the file system.

move_file Move a random file within a volume.

change_file Changes the content of a random file. A random number between 1 and file size + 40960 of ASCII 'A' are written to the file at a random offset.

clone_file Clones a random file within a volume.

add_folder Creates a new folder with a randomly selected folder name.

remove_folder Removes a folder.

After 20% of the actions, the *fcntl.F_FULLFSYNC* system call is executed. Additionally after every 100 performed actions, the disk is detached and attached again, to enforce the file system to be written to disk.

All performed actions are saved in a human readable log format and in a ground truth format. The ground truth format stores information of different categories for later evaluation. The categories are taken from Carrier [4]:

file system item_id, item_type, status

file name volume, path, file name

metadata atime, mtime, ctime

content size, md5

The ground truth file contains entries for every item (file or folder) in every state (metadata and content) that was generated on the file system. Some of those states might be overwritten or have never been written to disk and are not recoverable.

4.2 Dataset Specifications

For evaluation of the developed approaches five images in different sizes have been created. All images contain four volumes, which is the default for an Apple macOS system. On each image, 1000 actions have been performed as described in Section 4.1. Table 13 lists the created images with the number of different actions performed.

5 EVALUATION AND COMPARISON

The results that are generated by the different approaches implemented in AFRO are tested against each other and against photorec, as an existing file carving tool, to assess whether those approaches can result in valuable results for forensic investigations compared to filecarving, and how the different approaches perform.

5.1 Evaluation of AFRO Approaches

To evaluate our three proposed and implemented methods (NXSB carving, APSB carving, node carving) as describes in Section 3, we compared the results of AFRO with the ground truth of the datasets which where created as described in Section 4.1. A summary of the results is given in Section 5.1.2.

5.1.1 Evaluation Process. To evaluate the results of AFRO, we compared the created output csv with the ground truth format we created during the dataset generation. Files are identified by file_id and volume. In case of the node carving, the resulting csv does not contain information about the volume. In this case, only the file_id is compared. If file_id and volume match, the file name, mtime, ctime, size and the md5 value are compared, to find out if the items are the same. These compared attributes can be grouped into different categories:

file name file name

metadata mtime, ctime

content size, md5

These categories are introduced in Carrier [4]. The atime attribute is contained in both, the result csv and the ground truth format, but it is ignored, as system processes like spotlight change the atime without being added to the ground truth format. The content category is not tested for folders.

5.1.2 Evaluation Results. Table 14 displays a summary of the evaluation. It lists the percentage of recovered files, where all information of the categories where fully recovered from the file system. Note that we consider each activity

image name	size	volume count	add file count	delete file count	move file count	change file count	clone file count	add folder count	remove folder count
image_2G_4	2 GB	4	168	158	161	145	145	178	45
image_5G_4	5 GB	4	158	171	153	148	152	171	47
image_10G_4	10 GB	4	160	161	136	148	166	179	50
image_100G_4	100 GB	4	197	142	179	154	148	147	33
image_500G_4	500 GB	4	149	161	158	164	133	179	56

Table 13: Created Images

that was performed (file modifications and so on) to result in an additional recoverable file as an upper boundary of recoverable files, because of the copy on write strategy of APFS. Parsing the file system always recovered all files which were not deleted, which shows that the parsing algorithm for the images is correct. Thus the 14.1% on average denote the number of files that were present on the final state of the file system. Each percentage that lies above the percentage in the *parsing* column means that deleted or prior versions of files have been recovered in addition. Because many activities have been performed in a short timeframe during the generation of the dataset and in this case, not every single change is committed to disk as we realized during the evaluation. Some changes are only made in memory and are not flushed to the disk. It is therefore practically impossible to recover all file states that existed and reach the 100%, but this would be the absolute upper bound of recoverable files and versions of files and thus we take this number as benchmark. In one case, image_2G_4, only the active files were retrieved using carving container superblocks. In all other cases carving container superblocks resulted in more recovered data than parsing. Carving for volume superblocks recovered in all test cases more files than the two methods evaluated before. Changed, moved and deleted file were successfully recovered. Carving nodes restored less full files in all but one case (image_500G_4) compared with volume superblock carving. While additional file names and meta-data were extracted, less content was correctly extracted. Nodes are structured below volume superblocks in the file system, so at this level there is no notion of volumes available. File_ids and extents_ids are only unique within a volume and can be the same between volumes. The connection of inode entries and extent entries is therefore ambiguous. While the test case of image_500G_4 shows, that carving for nodes has the potential to recover more files than carving for volume superblocks, the carving for volume superblocks generally was the approach with the highest recovery rate of files.

5.1.3 Runtime. Table 15 displays the runtimes of the different methods. All tests were executed on an *external* 1TB

image name	parsing	NXSB carving	APSB carving	node carving
image_2G_4	15.3%	15.3%	50.7%	48.6%
image_5G_4	13.1%	18.8%	45.2%	43.0%
image_10G_4	14.3%	21.4%	27.1%	26.8%
image_100G_4	15.6%	24.2%	27.3%	27.1%
image_500G_4	12.0%	19.0%	25.7%	27.2%
all	14.1%	19.7%	35.2%	34.5%

Table 14: Full matches for recovered files

SanDisk Extreme 500 SDD USB 3.0 drive (read up to 440 MB/sec, write up to 440 MB/sec) and a MacBook Pro (2,3 GHz Dual-Core Intel Core i5 (Turbo Boost up to 3,6 GHz) with 64 MB eDRAM, 8 GB 2133 MHz LPDDR3). The runtime of the parsing of all test images is constant. All images contain about the same number of files and folders and only those structures need to be read, so there is no rise in runtime. The other methods have a linear increment of the runtime for the images of different sizes. Checking all blocks for structures increases time with the image size, which explains this increase. The runtime of all test runs is within a reasonable time for a forensics investigation.

image name	parsing	NXSB carving	APSB carving	node carving
image_2G_4	0:01	0:08	0:09	0:11
image_5G_4	0:01	0:18	0:21	0:23
image_10G_4	0:01	0:34	0:33	0:35
image_100G_4	0:01	5:34	5:27	5:31
image_500G_4	0:01	41:36	41:04	42:16

Table 15: Runtime (min:sec)

5.2 Comparison of AFRO with File Carving

To further evaluate our approaches we compared our best approach, APSB carving, against file carving. File carving recovers files using their headers and footers. Specific file signatures are searched to find those structures. The limitation of this approach is not the file system, but the file types that are known to the carver. Unknown file types cannot be recovered. We used *photorec* by Grenier [8] to compare our best approach with a file carving tool. We chose *photorec* as it achieved the best file carving results in different studies [1, 11]. Most metadata cannot be recovered by *photorec*, which is an advantage of our approach. Table 16 displays the percentage of content that was recovered by APSB carving compared with content recovered by *photorec*. In every tested image the carving for apsb returned a significantly higher amount of successfully recovered file contents. Additionally APSB carving was done faster than the carving by *photorec*.

image name	recovery rate		runtime (min:sec)	
	APSB carving	photorec	APSB carving	photorec
image_2G_4	54.9%	11.0%	0:09	0:15
image_5G_4	50.6%	8.0%	0:21	0:42
image_10G_4	52.0%	11.2%	0:33	1:11
image_100G_4	55.4%	10.4%	5:27	15:30
image_500G_4	44.7%	6.5%	41:04	74:25
all	42.6%	8.1%		

Table 16: Comparison with photorec

6 SUMMARY AND CONCLUSION

In this paper, we introduced different approaches to forensic file recovery, implemented them for evaluation and comparison, and obtained good results that outperforms existing approaches with a reasonable runtime performance.

6.1 Limitations

While the developed methods yield in good results, some assumptions about the file systems were made: Other versions of APFS were not addressed in this paper and might require adaption in the parser. Another limitation is that only the five datasets were tested. Although when generating the dataset as described we aimed at a representative evaluation, there might exist cases, which we are not yet aware of, that result in different internal structures, which our prototype would most probably fail to parse correctly. Furthermore, there are various specific features of APFS, for which further

research has to be done and that have not been in the focus of this work:

- Snapshots
- Encryption and compression
- Corrupted images (e.g. when not unmounted properly)

6.2 Outlook and Future Work

Besides testing and analyzing the points mentioned in Section 6.1, other improvements can be made, that we want to address in future work: The node carving process could be extended, so that extent entries can be correlated by their size and offsets and those groups can be linked by their range to the size of an inode. There is still a possibility for duplicates, e.g. files that have the same `file_id` and size, but exist on different volumes, but accuracy of the node carving method would improve. Another idea is to combine the different approaches. For example it is an option to first parse the file system and afterwards carve the remaining blocks for volume superblocks. This procedure can be recursively iterated for all methods. Additionally, AFRO could be used as a library for other forensic tools like the Sleuthkit or *dfvfs*. Rewriting AFRO and processing data with another language than python could improve the runtime.

6.3 Conclusion

In this work, we presented different novel approaches for forensic file recovery for APFS by interpreting different internal structures and implemented all of those approaches in a prototype we named AFRO, which presents the first open source forensic file recovery software for APFS at the time of writing, that recovery files not referenced from the file system. By using AFRO, we evaluated and compared the different approaches amongst each other and identified the method that so far delivers the best results and compared it to *photorec*. This showed that AFRO outperforms *photorec* on the evaluated APFS dataset and we identified several points for further improvement of our prototype, for example by combining different approaches.

REFERENCES

- [1] Khawla Alghafli, Andrew Jones, and Thomas Martin. 2014. Investigating and Measuring Capabilities of the Forensics File Carving Techniques. In *Future Information Technology*, James J. (Jong Hyuk) Park, Ivan Stojmenovic, Min Choi, and Fatos Xhafa (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 329–336.
- [2] Apple Inc. 2017. Apple File System Guide. https://developer.apple.com/library/content/documentation/FileManagement/Conceptual/APFS_Guide/Introduction/Introduction.html (last visited: 2018-01-10).
- [3] Orlando Bassotto. 2017. APFS lib. <https://github.com/tienex/apfs/> (last visited: 2018-04-15).
- [4] Brian Carrier. 2005. *File system forensic analysis*. Addison-Wesley Professional, USA.

- [5] A. Dewald and J. Plum. 2018. APFS INTERNALS FOR FORENSIC ANALYSIS. https://static.ernw.de/whitepaper/ERNW_Whitepaper65_APFS-forensics_signed.pdf (last visited: 2018-06-20).
- [6] EDRM. 2009. EDRM File Formats Data Set 1.0. <https://www.edrm.net/resources/data-sets/edrm-file-format-data-set/> (last visited: 2018-01-18).
- [7] Simon Gander. 2017. APFS FUSE Driver for Linux. <https://github.com/sgan81/apfs-fuse/> (last visited: 2018-04-15).
- [8] Christophe Grenier. 2011. Photorec. <http://www.cgsecurity.org/wiki/PhotoRec>.
- [9] Kurt H Hansen and Fergus Toolan. 2017. Decoding the APFS file system. *Digital Investigation* 22 (2017), 107–132.
- [10] John Kodis. 1992. Fletcher’s Checksum. *Dr. Dobbs’s J.* 17, 5 (May 1992), 32–38. <http://dl.acm.org/citation.cfm?id=135011.135013>
- [11] Thomas Laurenson. 2013. Performance Analysis of File Carving Tools. In *Security and Privacy Protection in Information Processing Systems*, Lech J. Janczewski, Henry B. Wolfe, and Sujeet Sheno (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 419–433.
- [12] Paragon Technologie GmbH. 2017. APFS for Windows by Paragon Software. <https://backstage.paragon-software.com/home/apfs-windows/> (last visited: 2018-04-15).
- [13] SysDev Laboratories GmbH. 2017. Recovery Explorer. <http://r-explorer.com/technical.php> (last visited: 2018-04-15).
- [14] Thomas Tempelmann. 2017. Biskus APFS Capture. <http://biskus.com/> (last visited: 2018-04-15).